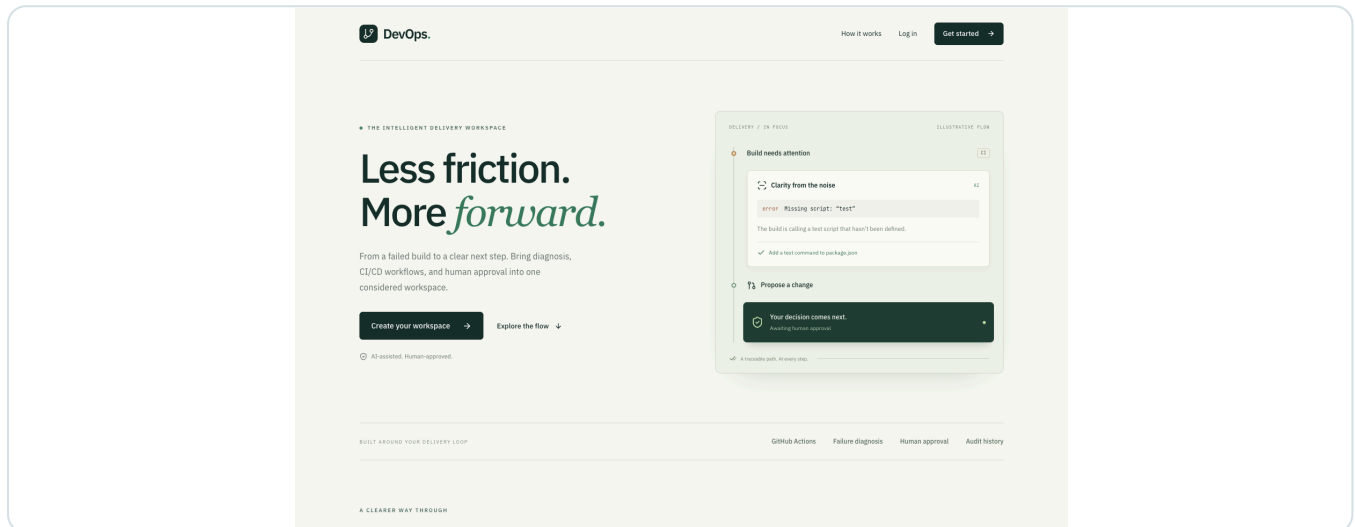


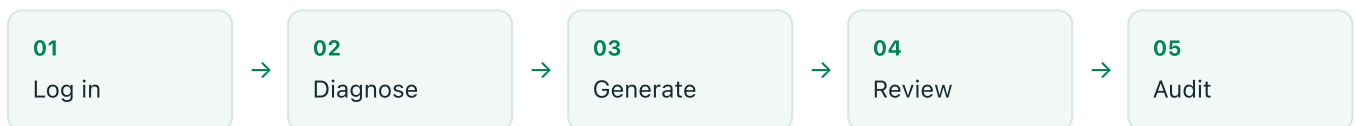
YOUR LIVE DEMO / 12 MINUTES

From a failed build to a controlled next step.

A practical demonstration guide for the Agentic AI-Powered Smart DevOps Assistant for Autonomous Software Delivery and Infrastructure Management.



The public landing page introduces diagnosis, workflows, and human approval.



WHAT TO SAY

"This project helps developers understand CI/CD failures and prepare delivery workflows. It routes tasks to a specialized agent, keeps repository changes behind human approval, and records who performed each action."

The main demonstration

Use Developer first, then Admin. Show diagnosis, YAML generation, agent routing, real failed-run records, approval rejection, and team oversight.

The connected extension

If GitHub credentials and a reachable webhook are ready, add real log download and an approved pull request. AI chat is an optional provider-backed feature.

How to use this guide

Each feature has click steps, the evidence to point at, and words you can say. Pages 2–3 are your rehearsal checklist; page 12 is your recovery and question sheet.

PREPARE / BEFORE THE SUPERVISOR ARRIVES

Set up a reliable demo.

Prepare the local route first. Keep connected services as an extension to the core presentation.

Start from the project root

Terminal 1 — backend

make backend

Terminal 2 — frontend

make frontend

If dependencies are missing, run `make setup` first. Leave both terminals running. If the services are already running, use those processes.

Check	Open / expected result
Website	http://localhost:5173/ · landing page opens
Backend	http://localhost:8000/api/v1/health/status · responds
API reference	http://localhost:8000/docs · Swagger UI opens

Use the two existing accounts

Role	Email	Presentation purpose
Developer	sachithasmadala@gmail.com	Personal work and approvals
Admin	malithiweerasooriya0@gmail.com	Team audit and member control

Enter the passwords privately. This document intentionally contains no passwords or tokens. Use separate browser profiles for the two accounts; alternatively, sign out before switching.

Available in the current local setup	Requires additional connection
Login, role visibility, model diagnosis, generated YAML, agent routing	AI-provider credentials for Agent chat
Saved failures; public-run import with internet; real pasted-log diagnosis	GitHub access for private repositories and automatic log download
Approval requests, rejection, audit, member management	Valid GitHub access for approved repository writes; reachable webhook for automatic events

Rehearse once

Confirm both logins, click Predict Failure and Generate Workflow, and open Workflow failures. Keep a genuine GitHub job log and its run URL ready if you want to demonstrate a saved diagnosis. Use web mode with authentication enabled for the role comparison.

RUN OF SHOW / KEEP THIS PAGE BESIDE YOU

Your 12-minute presentation.

Follow a single story: a Developer prepares work, a human controls changes, and an Admin can trace the result.

Time	Account / screen	Evidence to show
0:00–0:40	Public · Landing	The problem and the delivery flow
0:40–1:30	Developer · Overview	Role assigned by the backend; personal workspace
1:30–3:00	Developer · CI/CD assistant	Failure label, confidence, suggested fix
3:00–4:15	Developer · CI/CD assistant	Detected stack, workflow path, generated YAML
4:15–5:30	Developer · Agent orchestration	Intent, exactly one selected agent, trace
5:30–7:00	Developer · Workflow failures	Real run ID, repository, owner, log availability
7:00–8:30	Developer · Approvals	Fresh pending request; reject it; no GitHub write
8:30–10:45	Admin · Audit log / Users & roles	Actor filter, team visibility, access controls
10:45–12:00	Admin · Evaluation / closing	Reported metrics and the supported MVP boundary

Screen names and direct links

- **Overview:** [/dashboard](#)
- **CI/CD assistant:** [/diagnosis](#)
- **Agent orchestration:** [/multi-agent](#)
- **Workflow failures:** [/workflow-failures](#)
- **Approvals:** [/approvals](#)
- **Audit log:** [/executions](#)
- **Admin tools:** [/users](#) · [/evaluation](#)

For a five-minute demo

Show Developer login → one diagnosis → generated YAML → one real failed-run record → a rejected approval → Admin audit. Describe optional GitHub execution rather than waiting for it live.

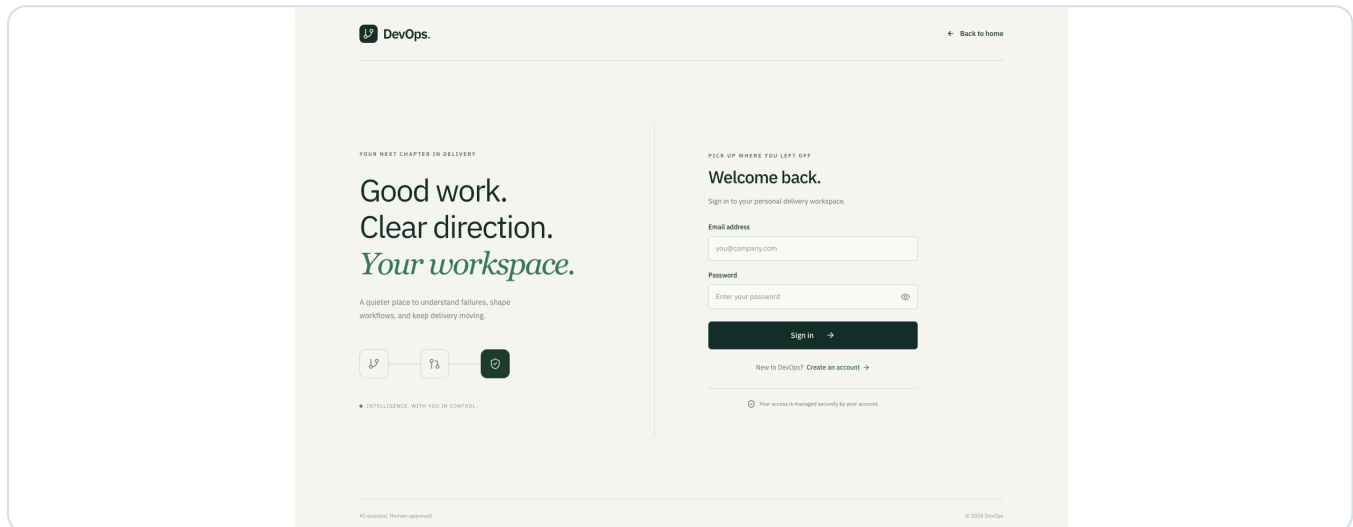
WHAT TO SAY

"I will first show the Developer's workflow. Then I will switch to Admin to show accountability and access management."

STEP 1 / LOGIN AND PERSONAL VISIBILITY

Start as the Developer.

Show that the account determines access and that members work in their own workspace.



The login form asks for credentials. It does not ask the user to choose a role.

- 1 Open the [landing page](#) and select **Log in**. Enter the Developer email and password, then click **Sign in**.
- 2 On **Overview**, point out the Developer role and personal activity. Counts can change as you rehearse.
- 3 Open **Audit log**. Show the heading **Your audit log** and that the entries belong to this account.
- 4 Explain that **Users & roles** and **Evaluation** are Admin screens. If you open `/users` as a Developer, the UI shows **Access restricted**.

WHAT TO SAY

"The backend assigns my role after authentication. As a Developer, I see my own records and authorized approval requests. The Admin can inspect activity across the team."

Optional signup preview

Open `/signup` in a logged-out window. Signup creates a Developer account without role selection. Use the existing accounts for the main demo; creating another Admin is not an available signup path.

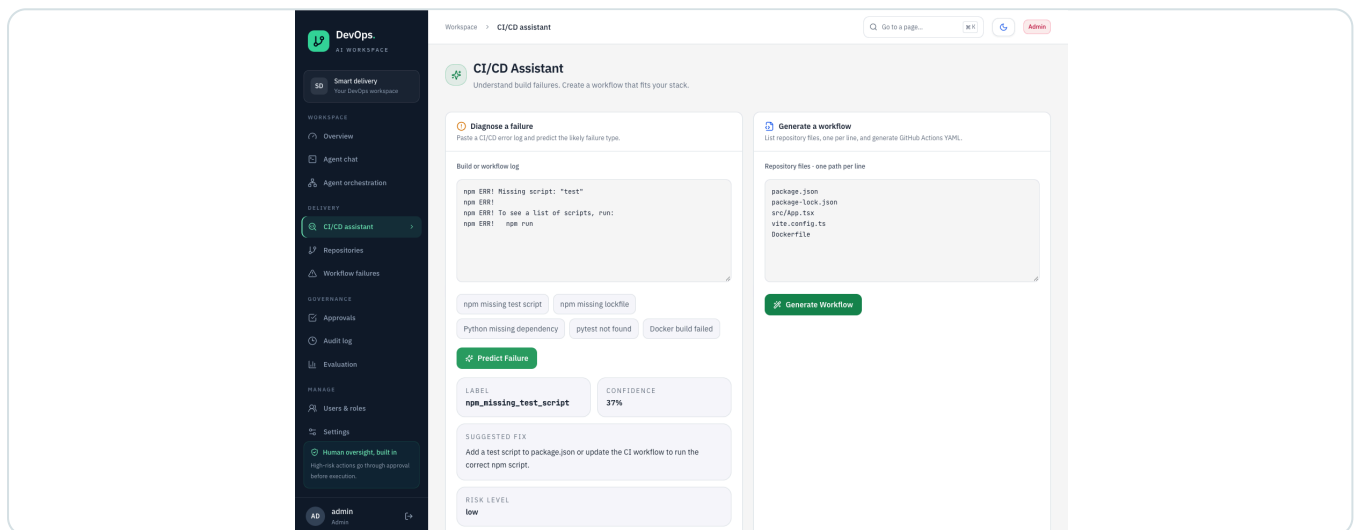
Proof: Developer workspace, personal audit heading, restricted Admin page. The later Admin comparison makes the ownership rule visible.

STEP 2 / TRAINED FAILURE CLASSIFIER

Diagnose a sample CI failure.

Navigation: CI/CD assistant · /diagnosis · Allow about 90 seconds.

- 1 Open **CI/CD assistant**. Under **Diagnose a failure**, choose the sample button **npm missing test script**.
- 2 Point to **Build or workflow log**, then click **Predict Failure**.
- 3 Show the predicted label `npm_missing_test_script`, the confidence, and the suggested fix. Read the score actually shown.
- 4 If time allows, choose **pytest not found** and predict again to show a second input. Return to the npm example for continuity.



Live local example: the trained model returns a label, confidence, and recommendation. Screen capture uses Admin; Developer has the same tool.

WHAT TO SAY

"This is a demonstration log. The classifier uses TF-IDF text features and Logistic Regression to identify a likely failure category. It suggests a next step; a human still reviews whether the change is appropriate."

Interpret confidence correctly

The displayed percentage applies to this prediction. It is not the overall accuracy of the model. Short logs can produce a low score; present the result honestly and use more complete logs when available.

Proof: input → predicted label → confidence → suggested fix. This local classifier path does not need an LLM API key. If the model cannot load, run `make train-model` before the session and recheck the prediction.

STEP 3 / LOCAL WORKFLOW GENERATION

Turn file paths into CI YAML.

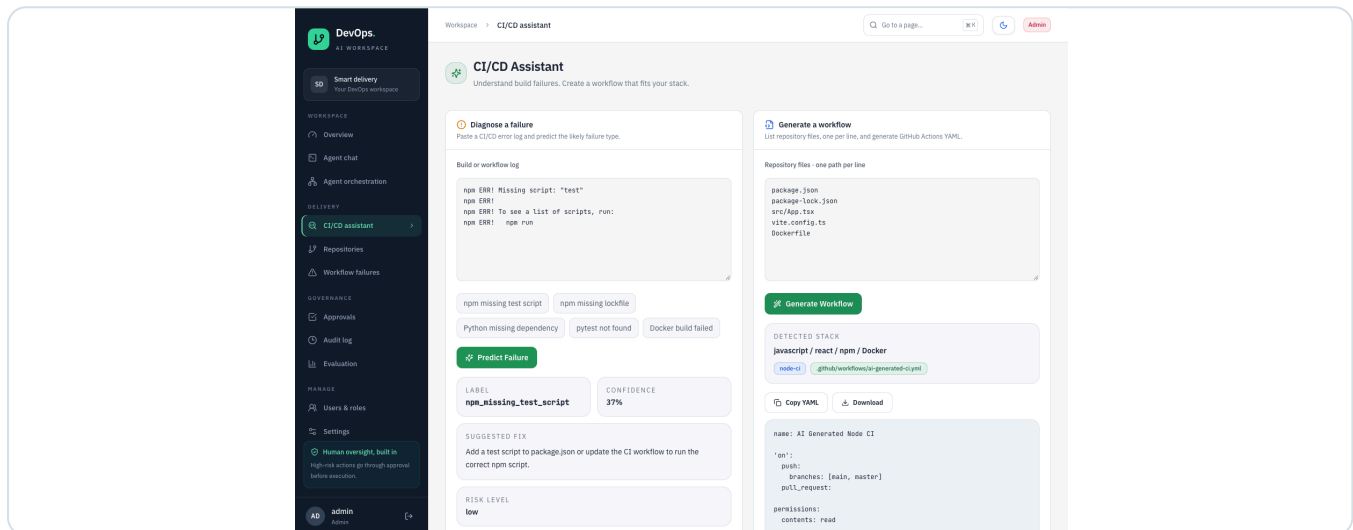
Navigation: CI/CD assistant · Generate a workflow · Allow about 75 seconds.

Paste one path per line

```
package.json
package-lock.json
src/App.tsx
vite.config.ts
Dockerfile
```

Then show the result

- 1 Click **Generate Workflow**.
- 2 Point to the detected stack and workflow path.
- 3 Show the YAML preview, **Copy YAML**, and **Download**.



A generated Node / React workflow and the download controls in the current application.

WHAT TO SAY

"The assistant detects the stack from the supplied repository paths and generates a suitable GitHub Actions workflow. At this stage I am preparing a file. Repository changes happen through a separately approved pull request."

What this proves

Point at stack detection, dependency-install and build/test steps in the generated YAML, and the output path `.github/workflows/ai-generated-ci.yml`. This example lists sample files; it is not a live repository scan.

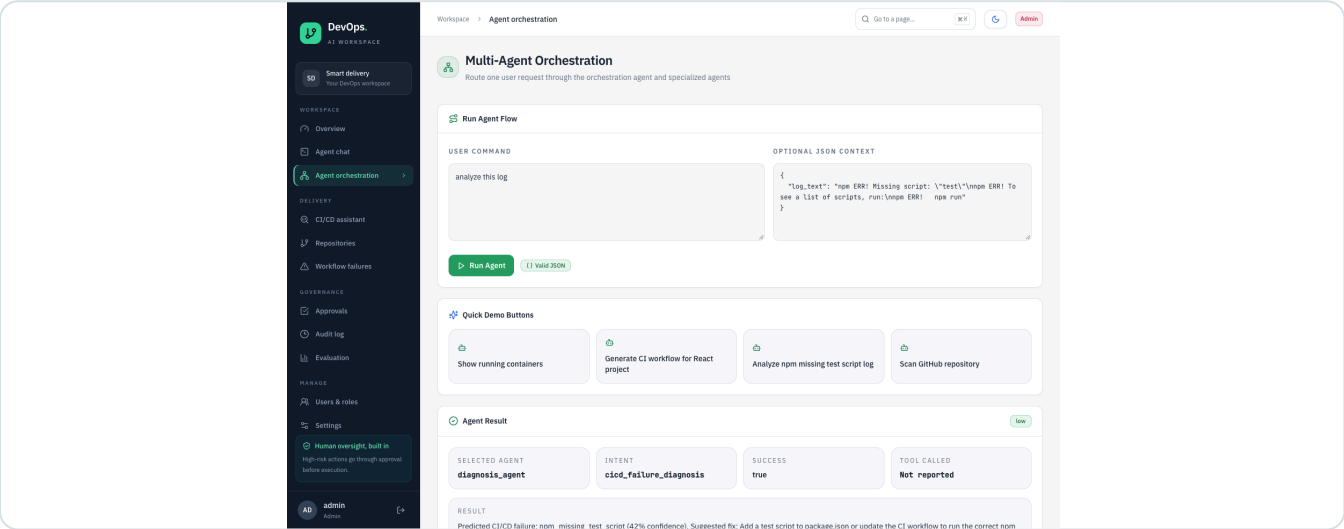
Optional connected route: the **Repositories** page offers **Scan Repository** and **Create Workflow PR**. Use that route only after GitHub access is verified; see page 11.

STEP 4 / ORCHESTRATION AND AGENT TRACE

Explain how the task is routed.

Navigation: Agent orchestration · /multi-agent · Allow about 75 seconds.

- 1 Select **Analyze npm missing test script log** from **Quick Demo Buttons**. The command and JSON context are filled for you.
- 2 Click **Run Agent**. Show **Selected Agent**: `diagnosis_agent`, the diagnosis intent, success, and risk.
- 3 Scroll to **Agent Trace**. Follow User → Orchestration Agent → Diagnosis Agent → model/service result.
- 4 For a second route, choose **Generate CI workflow for React project**, run it, and show `cicd_agent`.



The orchestration screen reports the selected specialized agent for a sample diagnosis.

Specialized agent	Responsibility
Diagnosis Agent	Classify logs and recommend fixes
CI/CD Agent	Detect stack and generate workflows
GitHub Agent	Repository and GitHub Actions integrations
CLI Agent	Allowlisted command / container inspection; optional demo

WHAT TO SAY

“The Orchestration Agent detects intent and selects exactly one specialized agent for the main task. The structured trace shows which route was selected and what happened. These local examples work without conversational AI credentials.”

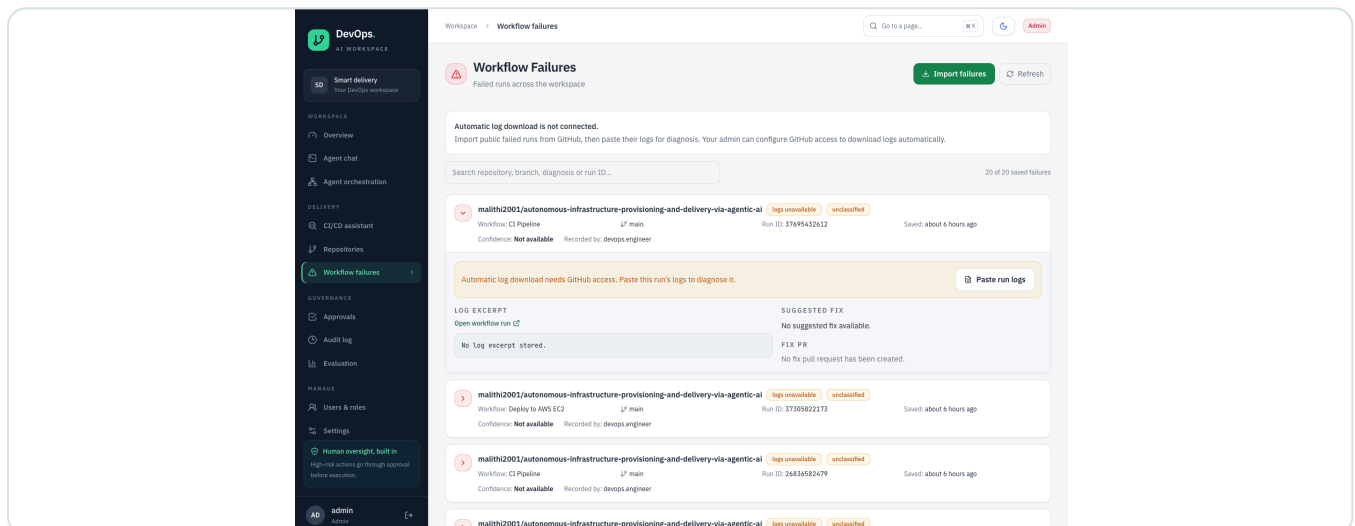
STEP 5 / REAL GITHUB FAILURE RECORDS

Show the failed runs clearly.

Navigation: Workflow failures · /workflow-failures · Allow about 90 seconds.

- 1 Open a saved run. Show the repository, workflow name, run ID, branch, status, and **Recorded by**. Use **Open workflow run** to show its actual GitHub page if internet is available.
- 2 To refresh historical runs, select **Import failures → Load from GitHub**. Enter the repository below and click **Load failed runs**.
- 3 Explain **logs unavailable**: the real failure metadata is saved, while automatic log download needs GitHub access. It is not a completed diagnosis.

Malithi2001/Autonomous-Infrastructure-Provisioning-and-Delivery-via-Agentic-AI



Saved real failed-run metadata. The local setup explains why logs are not automatically downloaded.

Optional: complete a saved diagnosis

From the failed job on GitHub, copy its **genuine error log**. Expand that same run in the app → **Paste run logs** → verify **Workflow run URL** → paste into **Failed run logs** → **Diagnose and save**. Show the saved label and fix recommendation. This path works without an LLM key.

WHAT TO SAY

"These are real GitHub runs. Each Developer sees their own imported records. Admin can see all owners. A run remains visible even when logs cannot be downloaded."

Keep demonstration evidence accurate

Use the built-in sample on the CI/CD assistant for sample diagnosis. Use genuine logs from the matching GitHub run for a saved workflow diagnosis. Repeated imports reuse that account's existing records; different accounts can each save the same run.

STEP 6 / HUMAN-IN-THE-LOOP CONTROL

Demonstrate approval by rejecting.

The local presentation can show request → review → rejection without GitHub credentials or repository changes.

- 1 As Developer, open **Agent orchestration**. In **User Command**, enter `create workflow pr`. In **Optional JSON Context**, paste the example below. It uses a placeholder repository for this rejected sample request.
- 2 Click **Run Agent**. Show that approval is required, with the proposed action and request ID. No PR has been created at this point.
- 3 Open **Approvals** immediately. Point to requester, repository, risk level, and action. Use the **fresh request you just created**.
- 4 Click **Reject**. Show the rejection feedback and the corresponding **Audit log** record. Rejection does not execute the GitHub action.

```
{  
  "repo_full_name": "owner/demo-repo"  
}
```

Request saved
Pending review



Human rejects
No execution



Decision logged
Auditable result

Demonstrate the ownership rule

Before rejecting, you can open Admin's **Approvals** queue in the other browser profile to show the Developer's request. Developer can review only their own authorized work; Admin can review team requests. User-access changes need Admin review.

If you already have a diagnosed failed run

Expand it in **Workflow failures** and choose **Request fix PR** instead. This also creates a saved approval. All fix PR writes require review, including low-risk fixes.

WHAT TO SAY

"The action is proposed first and waits for an explicit human decision. A Developer can review their own permitted work; they cannot approve another member's work or an Admin-only access change. I am rejecting this request to demonstrate that execution stops."

Approval is not successful execution

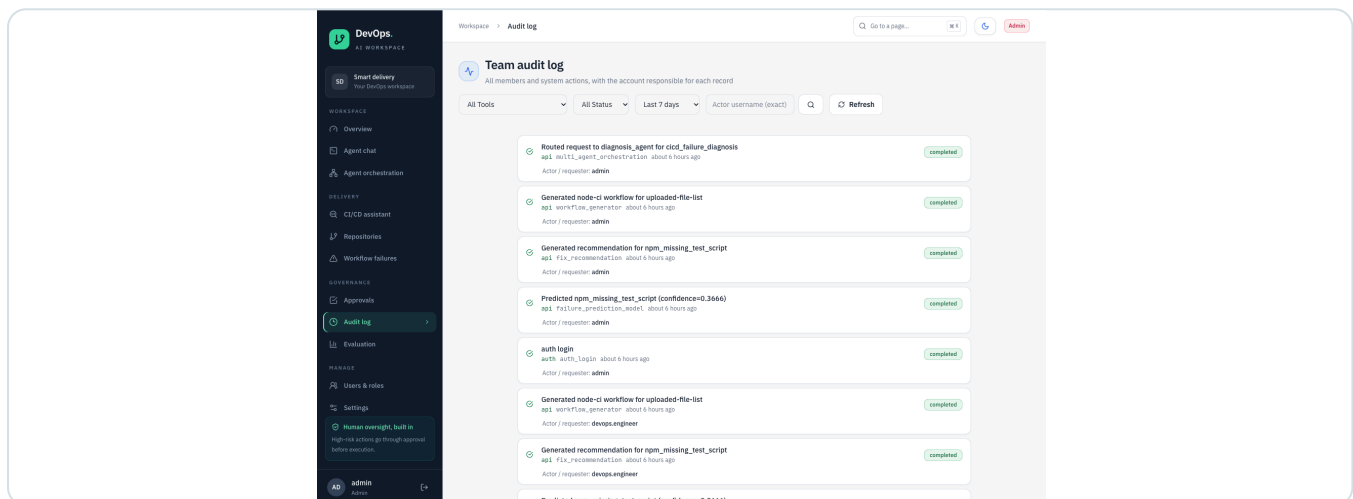
Approve only in the connected demo after reviewing the repository and proposed change. Then verify the execution result and actual PR URL. Expired requests cannot execute; create a fresh request when rehearsing.

STEP 7 / ADMIN OVERSIGHT AND MEMBER CONTROL

Show who did the work.

Switch to Admin · Audit log and Users & roles · Allow about two minutes.

- 1 Sign in as Admin. Open **Audit log** and show **Team audit log**, tool, status, actor, timestamp, and expandable details.
- 2 In **Actor username**, enter `devops.engineer` and apply the actor filter. Find the diagnosis, workflow, or approval activity from your demonstration. Confirm the username in the member list if the account has been renamed.
- 3 Clear filters. Contrast the team view with the Developer's **Your audit log**. Show that Admin can identify the responsible member.



Admin's team audit screen includes an actor filter and action history.

Show Activate / Deactivate

Open **Users & roles** (page title: **Members & access**). Show role and status filters, account status, and **Activate / Deactivate** controls. Use a disposable Developer member for an access-change demonstration.

Click **Deactivate** → review the saved request → **Reject change** for a demonstration that leaves access intact. For a full access-control demo, approve deactivation, show that the member loses access, then approve activation and show that they can sign in again. Existing sessions are revoked on deactivation.

WHAT TO SAY

"Admin sees team activity and can identify who performed an action. Access changes have their own approval and audit trail. The system also protects the active administrator from being removed accidentally."

Member creation

Add member creates a Developer. Public signup also creates a Developer. The application maintains the existing Admin account; it does not offer another Admin creation option.

OPTIONAL EXTENSION / CONNECTED GITHUB LOOP

Add live logs and a reviewed PR.

Use this extension only after a rehearsal succeeds with a repository you control.

Prepare before presentation

- Configure backend GitHub access through a GitHub App installation or the local `GITHUB_TOKEN` setting. Check access to the intended demo repository.
- For automatic diagnosis, make the backend webhook endpoint publicly reachable and configure a matching webhook secret. Keep secret values off the projector.
- Use a demo repository and a reviewed workflow change. For AI chat, separately configure the selected LLM provider and verify a harmless request before the session.

Stage	What to click / inspect	What confirms success
Repository analysis	Repositories → Scan Repository	Detected stack, existing workflows, readiness and warnings
Workflow proposal	Create Workflow PR → Review approval	Saved request and proposed target repository
Human decision	Approvals → Approve	Completed execution, not merely an approved status
GitHub review	Open the returned pull request URL	New branch, changed workflow file, reviewable PR
Failure ingestion	A completed failed run → webhook; or Import failures	Run metadata and actual downloaded logs when available
Diagnosis / fix	Request fix PR → Approvals	Stored prediction, recommendation, explicit review and result

Webhook endpoint:
`https://YOUR-DEMO-BACKEND/api/v1/webhooks/github`

Current local boundary

Local GitHub log downloads and AI chat are unconfigured. Public metadata imports and local model/YAML tools work. Demonstrate live logs, webhooks, and PR execution after observing them succeed.

WHAT TO SAY

"When connected to GitHub, the same review flow creates a branch and a pull request. The reviewer inspects the diff in GitHub. The app does not directly push changes to the default branch or automatically merge the PR."

Scope: this is a CI/CD MVP; cloud infrastructure remains future work.

RECOVERY / QUESTIONS / FINISH

Stay clear about what you proved.

Use this page if a service is unavailable or the supervisor asks about the project's limits.

What you see	What to do during the demonstration
No failure rows	Check the repository name, use Import failures, then Refresh. Developer sees only their own records; a diagnosis on the assistant page does not create a GitHub run.
Logs unavailable / unclassified	Show the genuine run metadata. Paste the matching real job log for diagnosis, or use the assistant's sample demo separately.
AI chat not configured	Continue with CI/CD assistant and Agent orchestration. The local classifier and YAML generation work without a conversational provider.
Expired approval / rejected request	Create a fresh review request. Refresh the failure list before requesting another fix review. Rejected and expired actions do not execute.
Approved, but execution failed	Inspect the audit result and GitHub access. Do not present the approval decision as a created PR.
Backend / browser cannot connect	Check the two local servers and their ports. If internet is down, use saved runs and local model/YAML examples.

Questions you can answer directly

What is the AI component? A trained TF-IDF / Logistic Regression failure classifier; optional LLM-backed chat. Task routing in the demonstrated orchestration path uses deterministic rules.

Why multiple agents? Each request is routed to one specialist for its main task. The specialists reuse existing services and tools.

How is access controlled? Backend role and ownership checks; Developer personal records and authorized own reviews; Admin team review and member management.

What does the confidence mean? The classifier's score for the selected label. Overall model performance is reported separately in Evaluation.

Evidence to keep ready

Admin → **Evaluation**: show the actual available dataset size, labels, accuracy/F1, and operational counts. Baseline verification from the preceding local check: **306 backend tests passed**, lint and production build passed, and both accounts passed live browser checks. Recheck before the demo; these results do not certify unconfigured external integrations.

WHAT TO SAY

"The demonstration proves authenticated access, local diagnosis, workflow generation, specialized-agent routing, real failed-run visibility, human review, and accountability. With GitHub connected, this extends to log downloads and reviewed pull requests."